ESTRUTURA DE DADOS

PROVAS - EXEMPLOS

CONCEITOS BÁSICOS

Algoritmo: sequências de passos para realizar uma tarefa

Estrutura de dados: forma sistemática de **organizar** e **acessar** os dados

Objetivo: escolher os melhores elementos para resolver um problema

- Eficácia: resolver problema
- Eficiência: utiliza a menor quantidade de recursos possíveis (memória e tempo)

COMPLEXIDADE DE ALGORITMOS

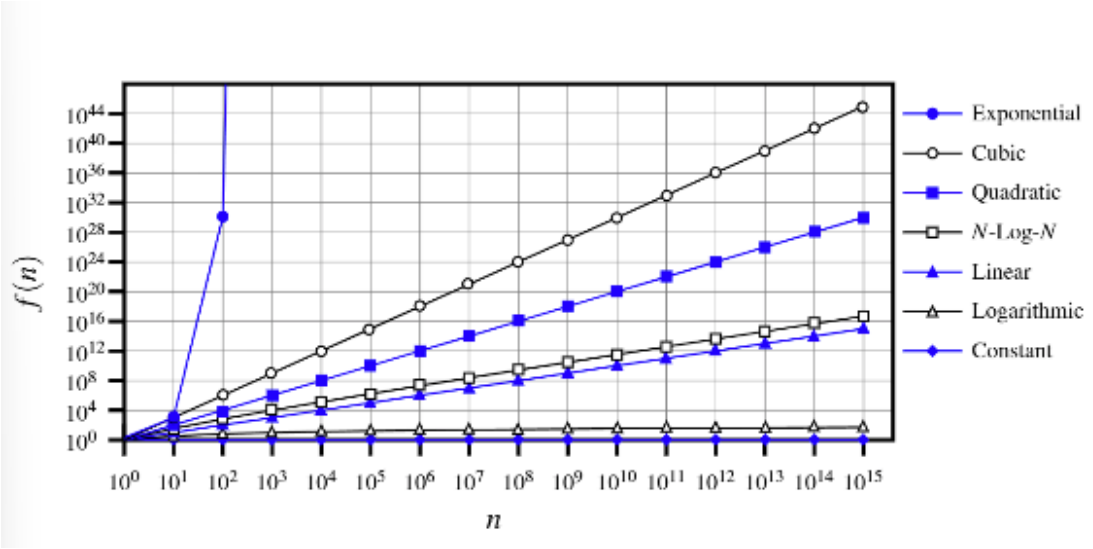
Um problema é caracterizado pela descrição de sua **entrada → instância** e **saída → solução/resultado**

> Melhor medida para definir bom, ou melhor: *Tempo de processamento + consumo de memória = análise de algoritmos (complexidade)*

Utilizamos métodos analíticos para definir a complexidade como uma função $f(n)$ do tamanho n da entrada

1. Contar o número de operações primitivas executadas pelo algoritmo;
2. Cada operação primitiva executa em um tempo **constante;**
3. Quanto menor for o número de operações mais eficiente é o algoritmo.

> Normalmente se **considera a complexidade no pior caso** e o processamento cresce na mesma proporção da dimensão de entrada (n).



Resumo de classes de complexidade (taxas de Crescimentos)

- Constante $O(1)$ O tempo não muda com a dimensão da entrada
- Logarítmica $O(logn)$: Crescimento muito lento; eficiente para grandes dados
- Linear $O(n)$: Cresce na mesma proporção da entrada
- Quadrática $O(n^2)$: Comum em laços aninhados
- Exponencial $O(2^n)$ Inviável para entrada muitos grandes

> Na análise assintótica, **funções constantes** são sempre as "mais lentas" em termos de taxa de crescimento. Para descobrirmos qual é a mais rápida, vamos olhar para os expoentes. Quando c**omparamos potências de n, quanto maior o expoente, mais rápido a função cresce**. Quando as funções têm o mesmo grau (como todas sendo n^1), nós olhamos para os coeficientes ou termos adicionais para ordená-las

	Grupo	O que analisar?	Exemplos
1º (lenta)	**Constante**	Não tem a variável n. Se for apenas um número, o crescimento é zero.	2^{10}, 1000, 1
2º	**Logarítmica**	Funções com $\log n$. Elas crescem muito devagar.	$\log n$, $\log(n^2)$
3º	**Linear**	Funções onde o n não tem expoente visível (n^1).	$2n$, $10n + 5$

4º	**Log-linear**	A mistura de n com $\log n$.	$n \log n$, $4n \log n$
5º	**Polinomial**	Funções onde n é a base e tem um expoente fixo.	n^2, n^3, n^{10}
6º (rápida)	**Exponencial**	Onde n está no **expoente**. Estas crescem absurdamente rápido.	2^n, 3^n, $n!$

Análise Assintótica

A análise completa (contagem de operação) é muito detalhada, além disso, o que importe é a **taxa de crescimento** da função de complexidade.

A análise assintótica foca em descrever a taxa de crescimento da complexidade de um algoritmo em função do tamanho n da entrada.

Exemplo prático: Em um algoritmo $O(n)$, se dobrarmos o tamanho da entrada, o tempo de execução também dobra. Já em um algoritmo $O(n^2)$, se dobrarmos a entrada, o tempo de execução é multiplicado por quatro.

> Concatenação de string, aumenta a complexidade, pois strings são imutáveis em Python, cada concatenação recria a string e copia os dados, gerando uma complexidade oculta quadrática $O(n^2)$ em vez de linear.

ESTRUTURAS FUNDAMENTAIS E LISTAS

Listas dinâmicas

É uma estrutura que **armazena elementos sequencialmente** e permite operações (inserção e remoção) em posições arbitrárias servindo de base para a construção de estruturas de dados mais sofisticados.

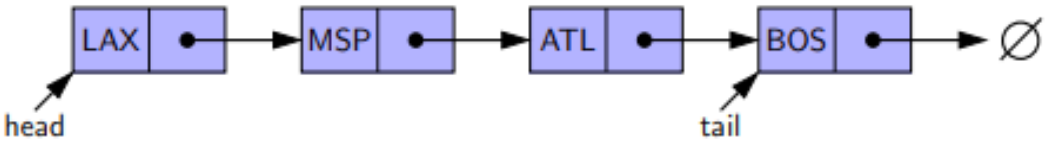
- Arranjos (Arrays): Alocações contíguas em memória (os elementos ficam grudados uns nos outros), com **acesso mais rápido** $[O(n)]$, mas tem **tamanho estático**, aumentar o tamanho/dobrar exige copiar todos os elementos. Já par **consultar ou atualizar** qualquer posição é instantâneo $O(1)$ e para **realocar** exige empurrar os elementos, para liber espaço, custando O(n)

Operação	Melhor caso	Caso médio	Pior caso	Observação
Acesso por índice	O(1)	O(1)	O(1)	Acesso direto pela posição
Busca sequencial	O(1)	O(n)	O(n)	Elemento no início / no fim
Busca binária	O(1)	O(log n)	O(log n)	Requer array ordenado
Inserção no início	O(n)	O(n)	O(n)	Desloca todos os elementos
Inserção no fim	O(1)	O(1)	O(n)	Pior caso: redimensionamento
Inserção no meio	O(n)	O(n)	O(n)	Desloca elementos posteriores
Remoção no início	O(n)	O(n)	O(n)	Desloca todos os elementos
Remoção no fim	O(1)	O(1)	O(1)	Sem deslocamento
Remoção no meio	O(n)	O(n)	O(n)	Desloca elementos posteriores
Remoção (Gerimundos)	O(1)	O(1)	O(1)	Troca com n-1 e remove o fim

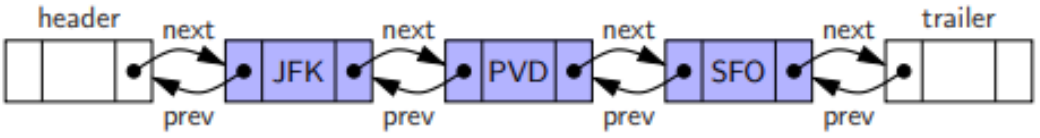
- Listas encadeadas: alocação dinâmica de memória (os elementos ficam espalhados) onde cada nodo guarda o dado e uma referência (um ponteiro) para o próximo. Para **inserir ou remover** nas pontas é instantâneo $O(1)$ basta mudar as referências sem empurrar ninguém, mas para acessar é lento, pois é preciso percorrer todos até encontrar o procurado $O(n)$

Operação	Melhor caso	Caso médio	Pior caso	Observação
Acesso por índice	O(1)	O(n)	O(n)	Não há acesso direto
Busca	O(1)	O(n)	O(n)	Percorre os nós sequencialmente
Inserção no início	O(1)	O(1)	O(1)	Atualiza somente o head
Inserção no fim	O(1)	O(1)	O(n)	O(1) se houver ponteiro tail
Inserção no meio	O(1)	O(n)	O(n)	Busca o nó predecessor
Remoção no início	O(1)	O(1)	O(1)	Atualiza somente o head
Remoção no fim	O(n)	O(n)	O(n)	Precisa chegar ao penúltimo nó
Remoção no meio	O(1)	O(n)	O(n)	Busca + ajuste de ponteiro

  - Simplesmente encadeada: Cada nodo tem o elemento e uma referência ao próximo. Remoção do final é lenta $[O(n)]$, ao exigir percorrer a lista até o penúltimo nodo



- Duplamente encadeada: cada nodo tem referência para o anterior $prev$ e o próximo $next$. Utiliza nodos sentinelas $header$ e $trailer$ para facilitar a implementação. Permite inserção e remoção nas extremidades em $O(1)$.



ALGORITMO DE BUSCA

Verificar se ele está em uma estrutura de dados. Ao realizar uma busca, o algoritmo pode retornar diferentes tipos de resultados, dependendo da necessidade: o próprio elemento, a posição onde ele se encontra, ou -1 caso o elementos não seja encontrado ou um valor lógico

Busca sequencial (Linear): Percorre a estrutura do início ao fim até encontrar o alvo. Pode ser usado em **arrays** ou em **linkedList** no pior caso sua complexidade é $O(n)$

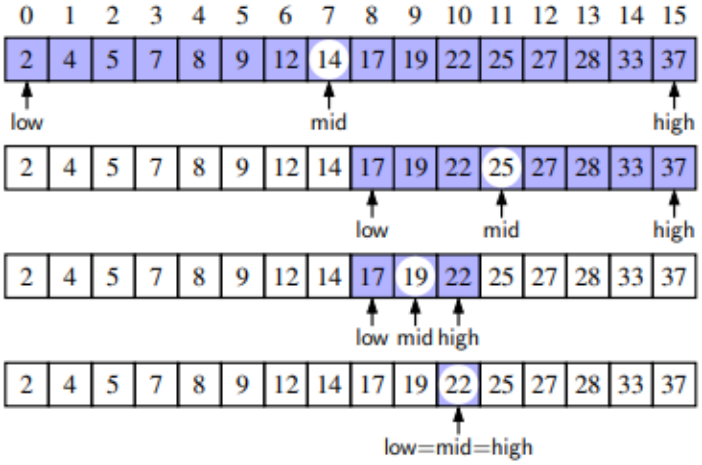
```
//BUSCA SEQUENCIAL EM ARRAY
def index(array, value):
    for i in range(len(array)): # [cite: 47]
        if array[i] == value: # [cite: 48]
            return i # [cite: 49]
    return -1 # [cite: 50]
```

Busca binária: Técnica eficiente para **arrays ordenados**. Complexidade $O(logn)$

Passo a Passo: Busca Binária

1. **Definir limites:** Inicia com `low = 0` e `high = tamanho - 1`.
2. **Calcular meio:** Encontra o elemento central `mid`.

3. **Comparar:**
    - Se `array[mid]` for o valor buscado, retorna a posição.
    - Se o valor buscado for maior que `array[mid]`, ignora a metade esquerda (`low = mid + 1`).
    - Se for menor, ignora a metade direita (`high = mid - 1`).
4. **Repetir:** Até encontrar o valor ou `low > high` (indicando que o elemento não existe na lista).




```python
//BUSCA BINÁRIA EM ARRAYS
def binary_search(array, value):
    low = 0 # [cite: 139]
    high = len(array) - 1 # [cite: 139]

    while low <= high: # [cite: 139]
        mid = (low + high) // 2 # [cite: 139]
        if array[mid] == value: return mid # [cite: 139]

        if array[mid] < value:
            low = mid + 1 # [cite: 139]
        else:
            high = mid - 1 # [cite: 139]
```

```
    return -1 # [cite: 139]
```

Algoritmo	Melhor caso	Caso médio	Pior caso	Espaço	Requer ordenado?
Busca sequencial	O(1)	O(n)	O(n)	O(1)	não
Busca sequencial (ordenada)	O(1)	O(n)	O(n)	O(1)	sim
Busca binária	O(1)	O(log n)	O(log n)	O(1)	sim

COMPLEXIDADE DOS ALGORITMOS DE ORDENAÇÃO

Ordenar consiste em rearranjar elementos conforme uma ordem (crescente ou decrescente). O desempenho ideal para o caso geral é $O(nlogn)$, como no **Quick Sort** e **Merge Sort**.

ALGORITMO	MELHOR CASO	PIOR CASO
Bubble sort	$O(n)$	$O(n^2)$
Insertion sort	$O(n)$	$O(n^2)$
Selection sort	$O(n^2)$	$O(n^2)$
Shell sort	$O(n)$	$O(n^2)$
Merge sort	$\mathcal{O}(n \log n)$	$\mathcal{O}(n \log n)$
Quick Sort	$\mathcal{O}(n \log n)$	$O(n^2)$
Radix sort	$O(n)$	$O(n)$

Qual é o melhor no cenário geral? O melhor desempenho para o caso geral é $\mathcal{O}(n \log n)$.

Por que usamos Quick Sort se o Merge Sort tem um pior caso melhor? O **Quick Sort** é o algoritmo mais utilizado na prática porque, além de ser mais rápido que o Merge Sort em execuções reais, o seu pior caso $\mathcal{O}(n^2)$ pode ser facilmente evitado escolhendo pivôs apropriados.

Por que não usar o Radix Sort sempre, já que é $\mathcal{O}(n)$? Apesar de ter

complexidade linear, ele **não é aplicável** em muitos casos gerais de comparação.

Passo a Passo: Insertion Sort (Ordenação por Inserção)

Ideia: Selecionar um elemento e inseri-lo na posição correta na parte já ordenada à sua esquerda.

1. **Percorrer:** Começa do segundo elemento até o final.
2. **Selecionar Chave:** Guarda o elemento atual (`key`) e define o índice do vizinho à esquerda.
3. **Comparar e Deslocar:** Enquanto o vizinho à esquerda for maior que a `key`, desloca esse vizinho para a direita.
4. **Inserir:** Coloca a `key` na posição vaga que mantém a ordem.
5. **Complexidade:** $O(n)$ no melhor caso (já ordenado) e $O(n2)$no pior caso (ordem inversa)

```
def insertion_sort(array):
    n = len(array) # [cite: 442]

    # O laço começa do índice 1 (segundo elemento), pois o
índice 0 já é considerado a sub-lista ordenada inicial.
    for i in range(1, n): # [cite: 442]
        key = array[i] # O elemento atual que vamos tentar
inserir na posição correta [cite: 442]
        j = i - 1 # O índice do elemento à esquerda da chav
e [cite: 442]

        # Enquanto não chegarmos no início (j >= 0) E o ele
mento à esquerda for maior que a chave:
        while j >= 0 and key < array[j]: # [cite: 442]
            array[j + 1] = array[j] # "Empurramos" o elemen
to maior uma posição para a direita [cite: 442]
            j -= 1 # Movemos nosso foco mais uma casa para
a esquerda [cite: 442]

        # Quando encontramos um elemento menor que a chave
```

```
(ou o início), inserimos a chave logo à frente dele.
        array[j + 1] = key # [cite: 442]
```

ORDENAÇÃO

Ordenar consiste em rearranjar elementos respeitando uma dada ordem (crescente ou decrescente). Qualquer coleção de itens pode ser ordenada, desde que sejam comparáveis (como números, strings ou atributos de livros, como título e autor). A eficiência é fundamental ao lidar com grandes volumes de dados.

Algoritmo	Melhor caso	Caso médio	Pior caso	Espaço	Estável?
Bubble sort	O(n)	O(n²)	O(n²)	O(1)	sim
Selection sort	O(n²)	O(n²)	O(n²)	O(1)	não
Insertion sort	O(n)	O(n²)	O(n²)	O(1)	sim
Merge sort	O(n log n)	O(n log n)	O(n log n)	O(n)	sim
Quick sort	O(n log n)	O(n log n)	O(n²)	O(log n)	não
Heap sort	O(n log n)	O(n log n)	O(n log n)	O(1)	não
Counting sort	O(n + k)	O(n + k)	O(n + k)	O(k)	sim

k = intervalo dos valores (counting sort). Quick sort tem pior caso O(n²) quando o pivô é sempre o menor/maior elemento array já ordenado sem aleatorização).

Insertion Sort - Ideia Geral

A cada iteração, pega o próximo elemento da parte não ordenada e o *insere na posição correta* dentro da parte já ordenada (à sua esquerda), deslocando os maiores para a direita.

Exemplo passo a passo — elemento 3 no vetor [2, 5, 8, 3, 9, 4, 1]

1. Pré-condição: [2, 5, 8] já ordenados.
2. Remove o 4º elemento (3) da posição.
3. Desloca o 8 → direita (8 > 3).
4. Desloca o 5 → direita (5 > 3).
5. Insere o 3 após o 2, pois 3 > 2.
6. Resultado: [2, 3, 5, 8] ordenados.

Não usa estrutura auxiliar — ordenação in-place.

Análise de Complexidade

- **Pior caso:** Ocorre quando a lista está em *ordem decrescente*. Cada elemento é comparado com todos os anteriores e inserido no início.

  Total de operações: 1+2+...+(n−1) = n(n−1)/2

- **Melhor caso:** Ocorre quando a lista já está *ordenada*. Cada elemento é comparado apenas com o último da sublista ordenada e permanece no lugar. Quanto mais ordenada, menor a complexidade prática.

```python
def insertion_sort(array):
    n = len(array)

    for i in range(1, n):
        key = array[i]     # elemento a inserir
        j = i - 1
        while j >= 0 and key < array[j]:
            array[j + 1] = array[j]  # desloca
            j -= 1
        array[j + 1] = key  # insere na posição correta
```

Selection Sort - Ideia Geral

A cada iteração, procura o **menor elemento** na porção não ordenada do vetor e o **troca** com o primeiro elemento dessa porção não ordenada. Com isso, a parte ordenada (no início do vetor) cresce um elemento de cada vez.

Exemplo passo a passo — ordenando o vetor [5, 3, 8, 4, 2]

1. **Passo 1:** O menor elemento de todo o vetor é `2`. Trocamos o `2` com o primeiro elemento (`5`).
   - Resultado: `[2, 3, 8, 4, 5]` (O `2` está ordenado).
2. **Passo 2:** Ignorando o `2`, o menor elemento do restante é `3`. Ele já está na posição correta (2ª posição), então fica onde está.
   - Resultado: `[2, 3, 8, 4, 5]` (`2, 3` estão ordenados).
3. **Passo 3:** Ignorando `2, 3`, o menor do restante é `4`. Trocamos com a 3ª posição (`8`).

- Resultado: `[2, 3, 4, 8, 5]` (`2, 3, 4` ordenados).

4. **Passo 4:** Ignorando `2, 3, 4`, o menor é `5`. Trocamos com a 4ª posição (`8`).
    - Resultado: `[2, 3, 4, 5, 8]` (Tudo ordenado).

Análise de Complexidade

- **Pior e Melhor caso:** Sempre será $O(n^2)$, pois ele sempre percorre todo o restante do vetor para encontrar o menor elemento, não importando se o vetor já está parcialmente ordenado.

Bubble Sort - Ideia Geral

Percorre o vetor várias vezes. Em cada passagem, compara **elementos adjacentes** (dois a dois) e os troca de lugar se estiverem na ordem errada. Como resultado, os maiores elementos "borbulham" rapidamente para o final do vetor.

Exemplo passo a passo — ordenando o vetor [5, 3, 8, 4, 2]

1. **Passagem 1:**
    - Compara `5` e `3`. Como 5 > 3, troca. → `[3, 5, 8, 4, 2]`
    - Compara `5` e `8`. Estão na ordem certa, não troca. → `[3, 5, 8, 4, 2]`
    - Compara `8` e `4`. Como 8 > 4, troca. → `[3, 5, 4, 8, 2]`
    - Compara `8` e `2`. Como 8 > 2, troca. → `[3, 5, 4, 2, 8]`

    (O maior elemento, `8`, foi parar na última posição. Agora ele está ordenado).

2. **Passagem 2:** (ignora a última posição que já tem o `8`)
    - Compara `3` e `5`. Não troca.
    - Compara `5` e `4`. Troca. → `[3, 4, 5, 2, 8]`
    - Compara `5` e `2`. Troca. → `[3, 4, 2, 5, 8]`

    (O `5` borbulhou para sua posição correta).

3. **Passagem 3:**
    - Compara `3` e `4`. Não troca.
    - Compara `4` e `2`. Troca. → `[3, 2, 4, 5, 8]`

4. **Passagem 4:**

- Compara `3` e `2`. Troca. → `[2, 3, 4, 5, 8]`

(Vetor totalmente ordenado).

Análise de Complexidade

- **Pior caso:** $O(n^2)$ quando o vetor está em ordem decrescente.
- **Melhor caso:** $O(n)$ se implementado com uma verificação para parar caso nenhuma troca seja feita em uma passagem inteira (vetor já ordenado).

PILHAS, FILAS E DECKS

- **Pilhas:** Política LIFO (Last in, First Out). O último a entrar é o último a sair
  - Push: insere no topo
  - Pop: remove e retorna do topo
  - Top: lê o topo sem remover

```
#USANDO ARRANJO
push(e):
  t += 1; data[t] ← e

top():
  retorna data[t]

pop():
  retorna data[t]; t -= 1
```

```
#USANDO ENCADEAMENTO
push(e):
  llist.add_first(e)

top():
  retorna llist.first()

pop():
  retorna llist.remove_first()
```

Operação	Melhor caso	Caso médio	Pior caso	Observação
push	O(1)	O(1)	O(1)	Insere no topo
pop	O(1)	O(1)	O(1)	Remove do topo
top / peek	O(1)	O(1)	O(1)	Apenas consulta o topo
is_empty	O(1)	O(1)	O(1)	Verifica se size == 0
Busca	O(1)	O(n)	O(n)	Não suportada nativamente

- **Filas:** Política FIFO (First In, First Out). O primeiro a entrar é o primeiro a sair
  - Enqueue: insere no final
  - Dequeue: remove e retorna do inicio
  - First: lê o início sem remover

  O desafio da fila com arranjo é o **dequeue eficiente**: remover do índice 0 com shift custa O(n). A solução é usar uma variável f como referência da frente, reutilizando o espaço de forma circular (*circular array*).

```
#ARRANJO CIRCULAR
enqueue(e):  O(1)
  pos ← (f+size) % len(data)
  data[pos] ← e

dequeue():    O(1)
  pos ← f
  f ← (f+1) % len(data)
  retorna data[pos]

first():      O(1)
  retorna data[f]
```

```
#ENCADEAMENTO
enqueue(e):
  llist.add_last(e)

dequeue():
  retorna llist.remove_first()
```

```
first():
  retorna llist.first()
```

Operação	Melhor caso	Caso médio	Pior caso	Observação
enqueue	O(1)	O(1)	O(1)	Insere no final
dequeue	O(1)	O(1)	O(n)	O(n) se implementado em array sem circular buffer — como na Q8
front / peek	O(1)	O(1)	O(1)	Consulta o primeiro elemento
is_empty	O(1)	O(1)	O(1)	Verifica se size == 0
Busca	O(1)	O(n)	O(n)	Não suportada nativamente

- **Deques:** Inserção e remoção em ambos os lados, mas nunca no meio
  - add_first: adiciona no início
  - add_last: adiciona no final
  - remove_first: remove do inicio
  - remove_last: remove do fina

```
#ARRANJO CIRCULAR
add_first(e):
  f ← (f-1) % len(data)
  data[f] ← e

add_last(e):
  pos ← (f+size) % len(data)
  data[pos] ← e
```

Operação	Melhor caso	Caso médio	Pior caso	Observação
push_front	O(1)	O(1)	O(1)	Insere no início
push_back	O(1)	O(1)	O(1)	Insere no fim
pop_front	O(1)	O(1)	O(1)	Remove do início
pop_back	O(1)	O(1)	O(1)	Remove do fim
front / back	O(1)	O(1)	O(1)	Consulta extremidades
is_empty	O(1)	O(1)	O(1)	Verifica se size == 0
Acesso por índice	O(1)	O(n)	O(n)	Não suportado em deque encadeado
Busca	O(1)	O(n)	O(n)	Não suportada nativamente

> Encadeamento — usa lista duplamente encadeada: Necessita de lista *duplamente* encadeada (com ponteiros next e prev) para permitir remoção eficiente em ambas as extremidades. Pilha e Fila usam lista *simples*. Deque usa lista *dupla*.

A Regra de Ouro da Complexidade (Decore isto):

Para Pilhas, Filas e Deques,**todas as operações principais devem ter complexidade** $\mathcal{O}(1)$. Isso vale para inserção, remoção, verificação de topo/início, tamanho `(size)` e se está vazia `(is_empty)`. Não importa se a implementação usa Arranjo (Array) ou Lista Encadeada (Linked List), o desempenho é sempre $\mathcal{O}(1)$ constante.